

Gradient descent & Back Propagation

Aim:-

To study & implement gradient descent & backpropagation in a deep neural network for optimizing weights and minimizing error.

Objective:-

1. To understand how gradient descent updates model parameters
2. To learn the role of backpropagation in computing gradients efficiently
3. To analyze the effect of weight updates on training accuracy.

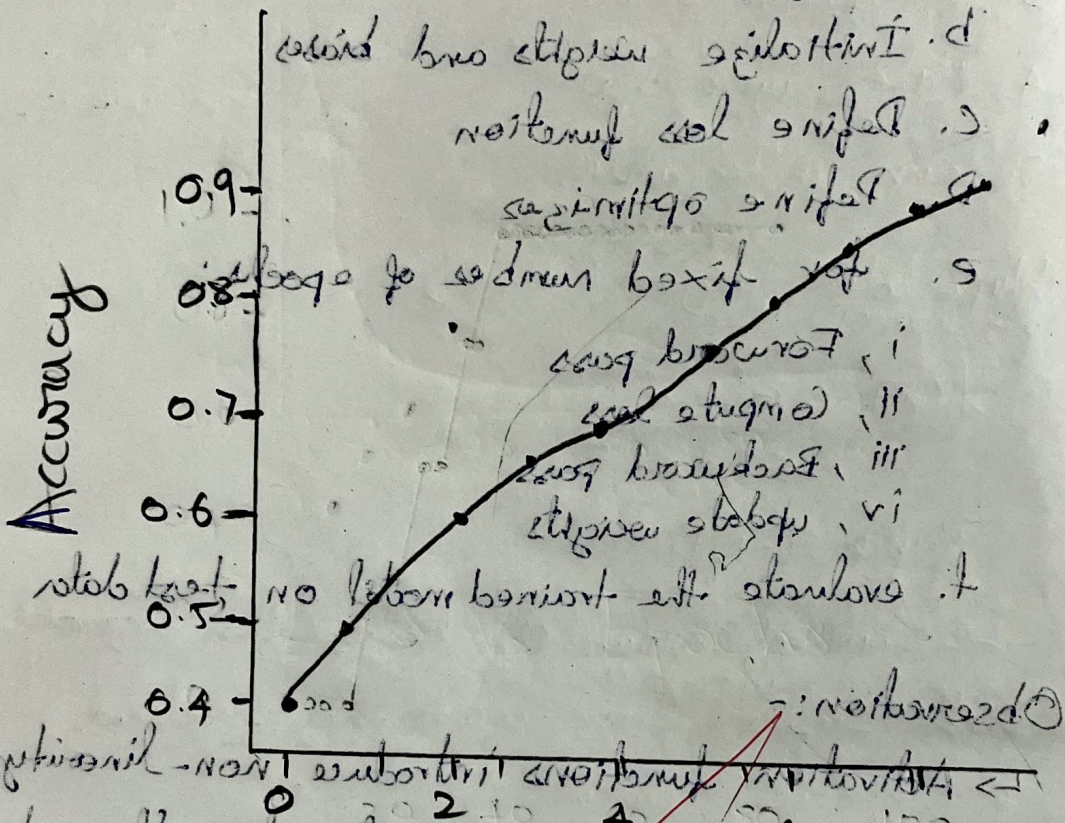
Pseudo Code:

1. Initialize input data (x) & target output (y)
2. Define network architecture,
 - Input layer
 - Hidden layer with activation function
 - Output layer
3. Randomly initialize weights & biases
4. Choose learning rate and number of epochs

Epochs Vs Accuracy

needed all the network activation and weights (sigmoid, tanh, ReLU, Leaky ReLU, etc.)

For each activation function, we assign the activation function to the network.

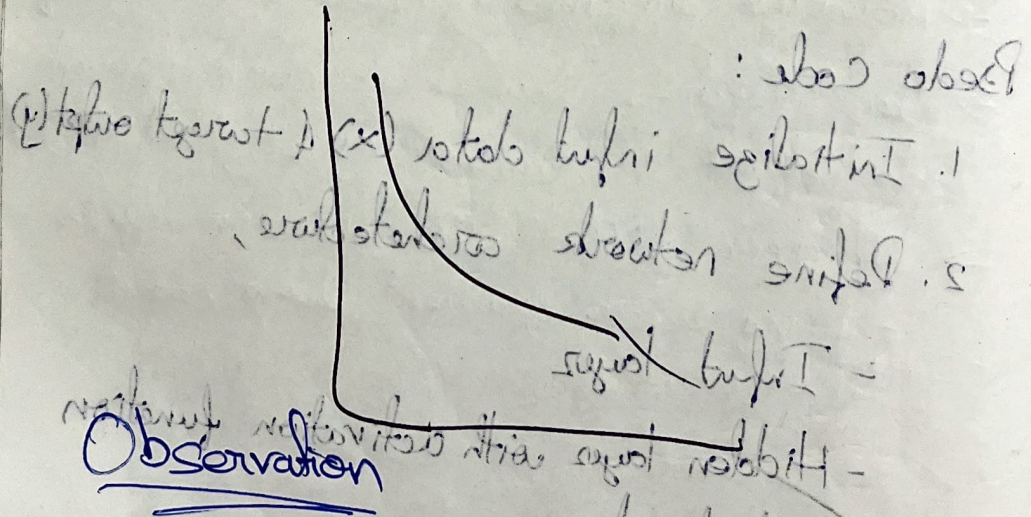
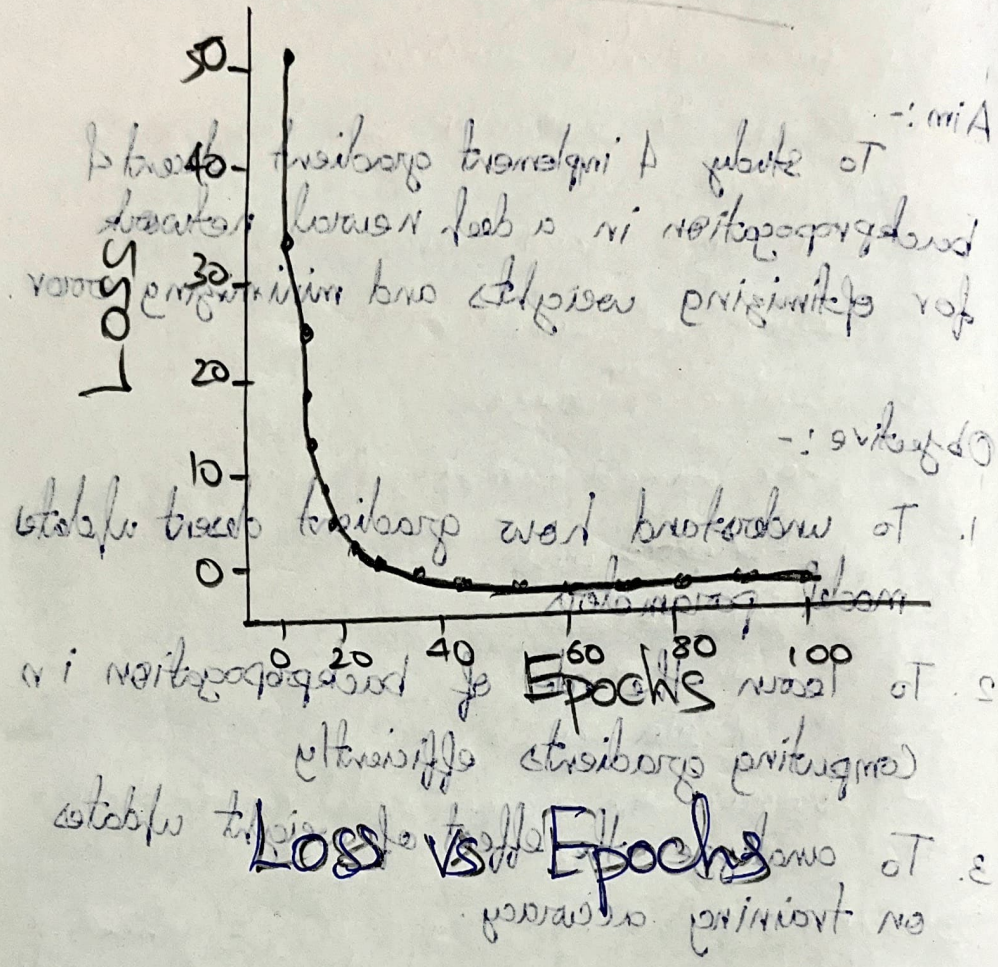


After training, we evaluate the model on a separate test set. The accuracy of the model is the percentage of test samples that the model correctly classifies.

Epoch is the number of times the entire dataset is used for training.

Epoch [100/1000]	Loss: 0.0232
Epoch [200/1000]	Loss: 0.0127
Epoch [300/1000]	Loss: 0.0070
Epoch [400/1000]	Loss: 0.0038
Epoch [500/1000]	Loss: 0.0021
Epoch [600/1000]	Loss: 0.0012
Epoch [700/1000]	Loss: 0.0006
Epoch [800/1000]	Loss: 0.0003
Epoch [900/1000]	Loss: 0.0002
Epoch [1000/1000]	Loss: 0.0001

Gradient Descent & Loss Propagation



Learned weight: 2.0085
Learned value: 0.9750

Handwritten notes:

- Output layer
- Hidden layer with activation function
- Input layer
- Define network architecture
- Initialize input color (x) & target output (y)
- Epoch Code:

5. Repeat for each epoch:

a. Forward propagation

- i, compute weighted sum for each layer
- ii, Apply activation function

b. Compute loss

c. Backward propagation:

- (i), Compute gradient of loss w.r. to output
- (ii), propagate error backwards through each layer
- (iii), calculate gradient for weights and biases

d. update parameters

$$\text{weight} = \text{weight} - \text{learning-rate} (\text{Gradient})$$

$$\text{bias} = \text{bias} - \text{learning-rate} (\text{Gradient})$$

6. Repeat until convergence (loss becomes small or max epochs reached)

7. Evaluate network performance on test data.

$$w_n = w - \frac{\partial L}{\partial w}$$

$$b =$$